

#	Finding	Classification	Justification
1	Syntax error	A – Should be fixed	A syntax error means Python cannot properly parse the code. This can prevent the affected module from running or being imported. There is no good reason to leave a syntax error in working code. Pylint reported an unterminated string in <code>admin.py</code> line 199.
2	Too many branches	A – Should be fixed	The function at <code>cli/main.py:124</code> has 16 branches , while Pylint's limit is 12. A large number of branches makes the function harder to understand and test. The function should ideally be divided into smaller functions. However, this is a design improvement rather than proof that the function is completely wrong.
3	Too many instance attributes	B – Context dependent	The class has 14 attributes , while the limit is 7. This can indicate that the class is doing too much. However, if the class represents an object that naturally contains many related pieces of information, having many attributes can be reasonable. Therefore, I would review the class before changing it.
4	Too many arguments	B – Context dependent	The function has 9 arguments , compared with a limit of 5. This can make function calls difficult to understand. However, some functions naturally need several values. If all 9 arguments are necessary and closely related to the function's purpose, the warning may be acceptable.
5	Unsubscriptable object	A – Should be fixed	This finding indicates a possible type-related programming error. Unlike a style warning, it may cause an error when the code runs. Therefore, the expression should be checked and corrected rather than simply ignored.
6	Broad exception	A – Should be fixed	Catching general <code>Exception</code> can hide unexpected errors. More specific exceptions should normally be caught so that the program handles known errors properly. The report shows this issue in <code>export_utils.py</code> .

7	Bare except	A – Should be fixed	A bare <code>except:</code> catches exceptions without specifying their type. This can hide problems and make debugging difficult. A specific exception should normally be used. Pylint reported this at line 88 of <code>export_utils.py</code> .
8	Protected member access	B – Context dependent	Accessing members such as <code>_id</code> from another class is normally discouraged because they are intended to be internal. However, there may be a valid reason in this project, especially if the model classes do not provide public getters/properties. Therefore, I would inspect the design before changing every occurrence.
9	Unused variable	A – Should be fixed	The variable <code>notification</code> is reported as unused. If it is truly not needed, it should be removed. If the programmer intended to use it, then something may be missing. Either way, the finding should be investigated and resolved.
1 0	Line too long	B – Context dependent	A long line mainly affects readability and formatting. A line of 133 characters can usually be split into smaller lines, but it does not necessarily indicate a functional problem. If splitting it makes the code less clear, the warning could reasonably be accepted.